

Eksamen Databaser V2025

Eksamen under tilsyn 3. juni 2025

Teller 100% av karakteren i emnet. Vurdering: Bestått / Ikke bestått.

For å oppnå bestått:

Kandidaten har oppnådd kravene til læringsutbytte og viser nødvendige kunnskaper, ferdigheter og kompetanse.

I emnet Databaser betyr det at studenten har oppnådd kravene til læringsutbytte innen **alle de tre hovedområdene** i emnet: SQL, modellering og normalisering.

Som angitt i emnebeskrivelsen, er alle trykte og selvskrevne tekster tillatt som hjelpemidler på eksamen i dette emnet.

Oppgave 1 – modellering

Du har fått oppdrag å lagre databasedesign til en serie-database til strømmetjeneste NETSERIX. Databasen skal inneholde informasjon om seriene og episodene samt brukere og logg over hvilke serier de har sett på. Brukerne kan også vurdere ulike episoder. Din oppgave er å lage en entitetsrelasjonsmodell (ER-modell) i oppstartprosjekt.

Det skal lagres følgende informasjon om seriene: navn til serie, en kort og beskrivende tekst, sjangre (som Krim, Mysterie, Komedie ...), og hvilket språk den er på. Informasjon om de forskjellige episodene skal inneholde tittel, en kort og beskrivende tekst, lengden i minutt, produksjonsdato og URL til videofilen. For brukerne lagres det navn, epost og passord. Brukerne kan gi terningkast 1 – 6 og en anmeldelse av ulike episoder. Det skal logges datoene for den siste gang for hver serie en bruker så på den.

Oppgave: Tegn en modell for din foreslåtte løsning. Du kan selv velge om du vil benytte kråkefot eller UML notasjon. Velger du kråkefot trenger du ikke skille mellom identifiserende og ikke-identifiserende forhold. (UML notasjon har uansett ikke skille på dette.) Modellen din skal inneholde:

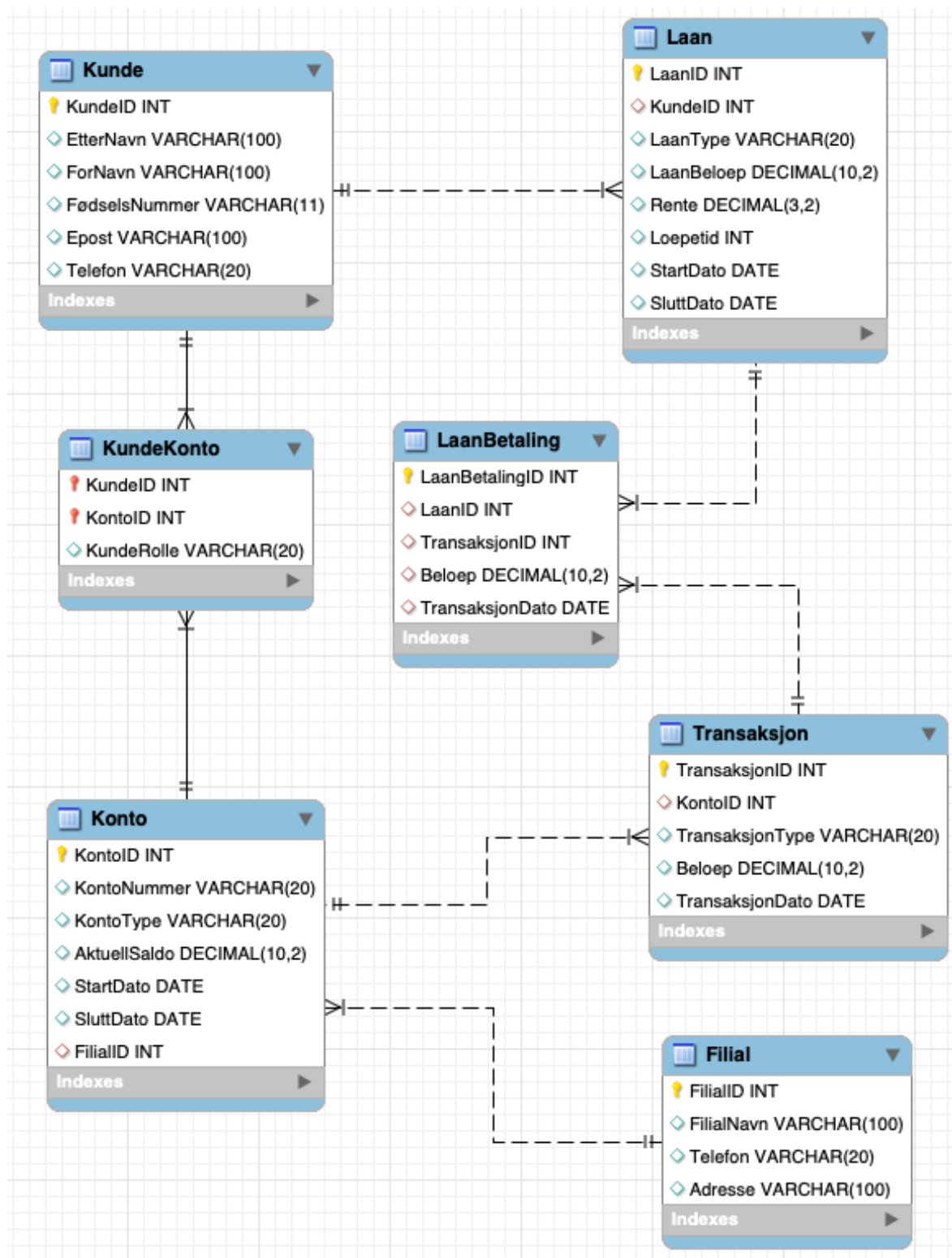
- Entitetene og deres attributter.
- Primærnøkler og fremmednøkler.
- Relasjonene mellom entitetene.
- Multiplisiteten (deltagelse og kardinalitet) for relasjonene.
- Hvis nødvendig, koblingsentiteter.

Hvis du synes noe er uklart, så gjør dine egne antagelser. Husk i så fall å gjøre rede for disse.

Vi anbefaler å tegne på ark, ta bilde og legge det inn i besvarelsen på WISEflow. Du kan også benytte tegnefunksjon i WISEflow og beskrive modellen med egne ord.

Oppgave 2 – SQL

Her ser du en ER-modell av en **Bank-database** som inneholder informasjon om bankkunder, bankkontoer, lån, transaksjoner og bankfilialer.



Kundetabellen inneholder nødvendig informasjon om kunde, samt fødselsnummer og adresse. Kontotabell har blant annet attributt kontotype som kan være «lønnskonto», «brukskonto», «lånkonto», og «sparekonto». Den inneholder også nåværende saldo, startdato når den ble åpnet og sluttdato om den ble avsluttet (sluttdato 9999-12-31 betyr at kontoen fortsatt er aktiv.) Attributt FilialID viser i hvilken filial kontoene ligger i.

En kunde kan være eier av en eller flere kontoer. En kunde kan også være disponent av en konto som eies av en annen kunde. Koblingsentitet KundeKonto gjenspeiler denne mange-til-mange relasjon mellom Kundetabell og Kontotabell.

En transaksjonstabell inneholder alle transaksjoner mot kontoene. Den har blant annet attributt TransaksjonsType som kan være «Innbetaling», «Utbetaling» eller «Overførelse».

I tillegg har kundene ett eller flere lån. Tabellen LaanBetaling inneholder nedbetalinger av lånet i form av transaksjoner fra lånkonto til lån.

I oppgavene nedenfor (2 a-h) skal du bruke denne Bank-databasen når du utformer SQL-uttrykkene dine. Husk at det er bedre å gjøre et forsøk på å besvare en oppgave enn å svare blankt. Vi foreslår at du bruker ca 2 timer for denne oppgaven. Hvis du står veldig fast på en oppgave, så kan det være lurt å gå videre til neste. Om nødvendig finner du på de to siste sidene av eksamensoppgaven ett vedlegg av de SQL-relevante begrep fra emnet.

- Skriv en SQL-spørring som sorterer kundene etter etternavn og deretter etter fødselsnummer.
- Skriv en SQL-spørring som henter ut all informasjon om kunder med etternavn som begynner på bokstaven "S".
- Skriv en SQL-spørring som henter alle kontonumrene for filialen med FilialNavn "Bergen Sentrum".
- Skriv en SQL-spørring som beregner summen av innbetalinger per LaanID.
- Oi, se her. Det har skjedd en feil i Konto-tabellen:

KontoID	KontoNummer	KontoType	AktuellSaldo	StartDato	SluttDato	FilialID
6	52982803926	Lånekonto	5173152.92	2001-01-14	9999-12-31	2
7	3514659809	Lønnskonto	122335.28	2006-06-16	9999-12-31	1
8	2114980157	Lånekonto	39600.20	2009-03-23	9999-12-31	5
9	46723432005	Brukskonto	99999999.99	2019-05-20	2024-08-25	5
10	5955674252	Sparekonto	64111009.03	2018-04-26	2020-04-10	2
11	40180700213	Brukskonto	99999999.99	2002-06-28	2011-12-25	4

Noen av de avsluttede kontoene viser AktuellSaldo lik 99999999.99 når det egentlig skulle være 0.00. Skriv en SQL-spørring som retter i alle de berørte radene.

- Skriv en SQL spørring som fjerner kolonne "Telefon" fra tabell Filial.
- Her mangler det noe! Følgende SQL- spørring skulle hente alle overførslers til kontoer knyttet til kunde Allison Anderson. Kan du fullføre SQL-koden slik at spørringen blir riktig?

```
SELECT a.EtterNavn
      , a.ForNavn
      , c.KontoNummer
      , d.TransaksjonID
      , d.Beløp
      , d.TransaksjonDato
FROM Kunde a
[ 🧩 SETT INN MANGLENDE KODE HER 🧩 ]
JOIN Konto c ON b.KontoID= c.KontoID
JOIN Transaksjon d ON d.KontoID = c.KontoID
WHERE [ 🧩 SETT INN MANGLENDE KODE HER 🧩 ];
```

- h) Åh. Det mangler enda mer. SQL-spørring skulle lagre viewet 'FastelInnbetalinger' som skulle vise gjennomsnittlig beløp og kontoid for transaksjoner av type "Innbetaling". Viewet skal kun vise kontoer med to eller flere innbetalinger. Den virker spenstige, eller?

```
[ 🧩 SETT INN MANGLENDE KODE HER 🧩 ]  
SELECT [ 🧩 SETT INN MANGLENDE KODE HER 🧩 ],  
KontoID  
FROM Transaksjon  
Where TransaksjonType = 'Innbetaling'  
GROUP BY KontoID  
[ 🧩 SETT INN MANGLENDE KODE HER 🧩 ];
```

Oppgave 3 – Normalisering

Tabellen nedenfor inneholder informasjon om spiller og spillekarakterer og deres våpner i en RPG (role-play-game) database.

SpillerID	SpillerNavn	SpillerPassord	Karakter	KarakterTypeOgAtferd	Våpen	VåpenType	VåpenBilde
1	Øyvind	IL0VeMM0rPg	Borirmir	Kriger god	Øks	Håndvåpen	øks.jpg
1	Øyvind	IL0VeMM0rPg	Borirmir	Kriger god	Sverd	Håndvåpen	sverd.jpg
2	NPC1	JustR0b0T1	Snaga	Ork ond	Hodeknuser	Håndvåpen	hodeknuser.jpg
3	Hilde	L0tRf0Re5r	Gandalf	Trollmann god	Tryllestav	Magiskvåpen	tryllestav.jpg
4	Marta	X\$5€#»21”	Aragon	Kriger god	Sverd	Håndvåpen	sverd.jpg
5	NPC2	JustR0b0T2	Snirk	Ork ond	Hodeknuser	Håndvåpen	hodeknuser.jpg
6	NPC3	JustR0b0T3	Saruman	Trollmann ond	Tryllestav	Magiskvåpen	tryllestav.jpg

Tabellen i denne oppgaven er et utdrag for å eksemplifisere hva som ligger i database-tabellen. Gjør dine egne antagelser om dataene i tabellen når du svarer på oppgavene nedenfor.

- Hva kan være primærnøkkel i tabellen?
- Vis to tilfeller av redundans i tabellen.
- Normaliser tabellen til tredje normalform. Du kan legge til kolonner hvis du ser behov for det.

Lykke til!

Vedlegg: MySQL, utvalgte datatyper/syntaks

[] angir valgfrie elementer og | angir alternativer.

Dette er en noe forenklet syntaks-oversikt som forhåpentligvis skal hjelpe dere i oppgavene på denne eksamen.

DATATYPER

CHAR[(lengde)]	Tekst med fast lengde, fra 0 til 255 tegn
VARCHAR (lengde)	Tekst med variabel lengde, fra 0 til 65 535 tegn
INT[(lengde)]	Heltall i området -2 147 483 648 til 2 147 483 647
ENUM (v1, v2...)	Definert verdisett
DATETIME	Tidsinformasjon som rommer både dato og klokkeslett.
DATE	Dato. Eks: '2016-12-06'
DECIMAL (A, B)	Desimaltall med A siffer og B desimaler . Eks: DECIMAL (5, 2) kan romme tall i området -999.99 - 999.99

FUNKSJONER

COUNT (*)	Antall
AVG (kolonne_navn)	Gjennomsnitt
SUM (kolonne_navn)	Sum
MIN (kolonne_navn)	Minimum
MAX (kolonne_navn)	Maksimum

OPERATORER

=	Lik (ikke m/wildcards!)
<> eller !=	Forskjellig fra
<	Mindre enn
>	Større enn
<=	Mindre eller lik
>=	Større eller lik
LIKE	Lik, godtar wildcards
IN	I gitt utvalg
BETWEEN	Utvalg, følges av AND
—	Wildcard, enkelt tegn
%	Wildcard, flere tegn
IS [NOT] NULL	Null / ikke null

LOGISKE OPERATORER

AND	Og
OR	Eller
NOT	Ikke

SYNTAKS

SELECT

```
SELECT      kolonnenavn [[AS] navn]
FROM        tabellnavn [[AS] alias]
[JOIN       tabellnavn [[AS] alias] ON felles nøkkel]
[WHERE      betingelse]
[GROUP BY   grupperingsuttrykk [HAVING betingelse]]
[ORDER BY   kolonnenavn]
```

VIEW

```
CREATE [OR REPLACE] VIEW viewnavn [(alias)]
AS kriterier
```

CREATE TABLE

```
CREATE TABLE tabellnavn
(
kolonnenavn datatype [UNIQUE|NOT NULL|DEFAULT|AUTO_INCREMENT|...],
...,
[[CONSTRAINT navn] PRIMARY KEY (kolonnenavn) ],
[[CONSTRAINT navn] FOREIGN KEY (kolonnenavn) REFERENCES tabellnavn (kolonnenavn) ]
)
```

ALTER TABLE

```
ALTER TABLE tabellnavn
[ADD kolonnenavn datatype],
[CHANGE kolonnenavn_nå kolonnenavn_ny datatype_ny],
[DROP COLUMN kolonnenavn]
```

DROP TABLE

```
DROP TABLE tabellnavn
```

INSERT INTO

```
INSERT INTO tabellnavn
VALUES (verdi1, verdi2, verdi3, ...)
eller
INSERT INTO tabellnavn (kolonnenavn1, kolonnenavn2, ...)
VALUES (verdi1, verdi2, ...)
```

UPDATE

```
UPDATE tabellnavn
SET kolonnenavn1 = verdi1, kolonnenavn2 = verdi2, ...
[WHERE betingelse]
```

DELETE FROM

```
DELETE FROM tabellnavn WHERE betingelse
```